



Introduction to Stock Market Simulator

A stock market simulator is a valuable tool for learning about stock trading without the risk of losing real money. It allows users to practice trading with virtual money and real-time market data, gaining valuable experience and insight into the stock market.

Presented by

Balla Sreeja Supreeti &

Amaze Sara Roy

Roll No: 23E41A3209 & 23E41A3201

CSBS



Benefits of Using a Stock Market Simulator

1

Risk-Free Learning

Simulators provide a safe environment to learn and practice trading strategies without the risk of financial loss.

2

Understanding Market Dynamics

Users can gain an understanding of market trends, patterns, and how different news events can impact stock prices.

3

No Financial Risk

It's an excellent way to experiment with investment techniques and build confidence without risking real money.

How a Stock Market Simulator Works

Virtual Trading Platform

The simulator provides a user-friendly platform where users can buy and sell stocks just like in a real trading scenario.

Real-Time Market Data

Simulators use real-time stock market data to ensure an authentic trading experience for users.

Interactive Learning

Users can learn about different financial instruments, such as stocks, options, and futures, through interactive simulations.

Features and Functionalities

Virtual Money

Users receive virtual money to start trading in the simulated environment.

Portfolio Management

Simulators allow users to build and manage a virtual portfolio of stocks and assets.

Market Orders

Users can place various types of orders, such as market, limit, and stop orders.

Pre-Requisites

Internet Access

Requires a stable internet connection to access the simulator and real-time market data.

Device Compatibility

The simulator may have specific compatibility requirements for various devices and operating systems.

Basic Financial Knowledge

Some understanding of stocks, trading, and investment is beneficial for using the simulator effectively.

HARDWARE REQUIREMENTS :

Processor: Minimum 1 GHz;
Recommended 2GHz or more
Ethernet connection (LAN) OR a wireless adapter (Wi-Fi)
Hard Drive: Minimum 32 GB; Recommended 64 GB or more
Memory (RAM): Minimum 1 GB; Recommended 4 GB or above

SOFTWARE REQUIREMENTS:

Windows Operating System (10 or Higher)
Turbo C++

Implementation

1

Choose a Simulator

Research and select a reputable stock market simulator that best suits your learning objectives.

2

Create an Account

Register an account and follow the setup instructions provided by the chosen simulator.

3

Begin Virtual Trading

Start practicing trading strategies using the virtual money provided by the simulator.

Source Code ...

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_STOCKS 10
#define INITIAL_BALANCE 10000

typedef struct
{
    char symbol[10];
    double price;
    int quantity;
}
    Stock;
typedef struct
{
    double balance;
    Stock stocks[MAX_STOCKS];
    int numStocks;
}
    Portfolio;
void initializePortfolio(Portfolio *portfolio)
{
    portfolio->balance = INITIAL_BALANCE;
    portfolio->numStocks = 0;
}

void displayPortfolio(Portfolio *portfolio)
{
    printf("\nPortfolio Summary:\n");
    printf("Balance: $%.2f\n", portfolio->balance);
    if (portfolio->numStocks > 0)
    {
        printf("Stocks:\n");
        for (int i = 0; i < portfolio->numStocks; ++i)
        {
            printf("%s - Quantity: %d, Price: $%.2f\n", portfolio->stocks[i].symbol,
```

```
portfolio->stocks[i].quantity, portfolio->stocks[i].price);
        }
    }
    else
    {
        printf("No stocks in the portfolio.\n");
    }
}

int buyStock(Portfolio *portfolio, char *symbol, int quantity, double price)
{
    if (portfolio->numStocks < MAX_STOCKS && portfolio->balance >= quantity * price)
    {
        // Check if the stock is already in the portfolio
        for (int i = 0; i < portfolio->numStocks; ++i)
        {
            if (strcmp(portfolio->stocks[i].symbol, symbol) == 0) {
                portfolio->stocks[i].quantity += quantity;
                portfolio->balance -= quantity * price;
                return 1;
            }
            // Successful purchase
        }

        // Stock not in the portfolio, add a new entry
        strcpy(portfolio->stocks[portfolio->numStocks].symbol, symbol);
        portfolio->stocks[portfolio->numStocks].price = price;
        portfolio->stocks[portfolio->numStocks].quantity = quantity;
        portfolio->numStocks++;
        portfolio->balance -= quantity * price; return 1;
        // Successful purchase
    }
}
```

```
    }
    else
    {
        printf("Error: Unable to buy the stock.\n"); return 0;
        // Failed purchase
    }
}

int sellStock(Portfolio *portfolio, char *symbol, int quantity, double price) {
    for (int i = 0; i < portfolio->numStocks; ++i)
    {
        if (strcmp(portfolio->stocks[i].symbol, symbol) == 0)
        {
            if (portfolio->stocks[i].quantity >= quantity)
            {
                portfolio->stocks[i].quantity -= quantity;
                portfolio->balance += quantity * price;
                if (portfolio->stocks[i].quantity == 0)
                {
                    // If no more stocks of this type, remove it from the portfolio
                    for (int j = i; j < portfolio->numStocks - 1; ++j)
                    {
                        portfolio->stocks[j] = portfolio->stocks[j + 1];
                    }
                    portfolio->numStocks--;
                }
                return 1;
                // Successful sale
            }
            else
            {
                printf("Error: Not enough stocks to sell.\n"); return 0; // Failed sale
            }
        }
    }
}
```


... *Source Code*

```
printf("Error: Stock not found in the\nportfolio.\n"); return 0;\n// Failed sale\n}
```

```
int main()\n{\n    Portfolio portfolio;\n    initializePortfolio(&portfolio);\n\n    char option;\n    char symbol[10];\n    int quantity;\n    double price;\n\n    do\n    {\n        printf("\nOptions:\n");\n        printf("1. DisplayPortfolio\n");\n        printf("2. Buy Stock\n");\n        printf("3. Sell Stock\n");\n        printf("0. Exit\n");\n        printf("Enter option: ");\n        scanf(" %c", &option);\n\n        switch (option)\n        {\n            case '1':\n                displayPortfolio(&portfolio);\n                break;\n\n            case '2':\n                printf("Enter stock symbol:\n");\n                scanf("%s", symbol);\n                printf("Enter quantity: ");\n                scanf("%d", &quantity);\n                printf("Enter price per share: $");\n                scanf("%lf", &price);\n                buyStock(&portfolio, symbol, quantity, price);\n                break;\n\n            case '3':\n                printf("Enter stock symbol:\n");\n                scanf("%s", symbol);\n                printf("Enter quantity: ");\n                scanf("%d", &quantity);\n                printf("Enter price per share: $");\n                scanf("%lf", &price);\n                sellStock(&portfolio, symbol, quantity, price);\n                break;\n\n            case '0':\n                printf("Exiting.\n");\n                break;\n\n            default:\n                printf("Invalid option. Please try again.\n");\n                }\n        }\n    while (option != '0');\n\n    return 0;\n}
```


... *Output*

Output:

Enter stock symbol: AAPL

Enter quantity: 5

Enter price per share: \$150.00

Expense added successfully.

Options:

1. Display Portfolio

2. Buy Stock

3. Sell Stock

0. Exit

Enter option: 1

Portfolio Summary:

Balance: \$9250.00

Stocks:

AAPL - Quantity: 5, Price: \$150.00

Options:

1. Display Portfolio

2. Buy Stock

3. Sell Stock

0. Exit

Enter option: 2

Enter stock symbol: GOOGL

Enter quantity: 3

Enter price per share: \$2000.00

Expense added successfully.

Options:

1. Display Portfolio

2. Buy Stock

3. Sell Stock

0. Exit

Enter option: 1

Portfolio Summary:

Balance: \$3250.00

Stocks:

AAPL - Quantity: 5, Price: \$150.00

GOOGL - Quantity: 3, Price: \$2000.00

Options:

1. Display Portfolio

2. Buy Stock

3. Sell Stock

0. Exit

Enter option: 3

Enter stock symbol: AAPL

Enter quantity: 2

Enter price per share: \$160.00

Stocks sold successfully.

Options:

1. Display Portfolio

2. Buy Stock

3. Sell Stock

0. Exit

Enter option: 1

Portfolio Summary:

Balance: \$3570.00

Stocks:

AAPL - Quantity: 3, Price: \$150.00

GOOGL - Quantity: 3, Price: \$2000.00

Options:

1. Display Portfolio

2. Buy Stock

3. Sell Stock

0. Exit

Enter option: 0

Exiting.

In this example:

The user starts with a balance of \$10,000 and an empty portfolio.
The user buys 5 shares of AAPL at \$150.00 per share.
The user buys 3 shares of GOOGL at \$2000.00 per share.
The user sells 2 shares of AAPL at \$160.00 per share.
The program displays the updated portfolio after each transaction.
The user exits the program.

REFERENCES

1. Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, Prentice Hall of India

2. E. Balagurucamy, Computer fundamentals and C, 2nd Edition, McGraw Hill

3. Herbert Schildt, C: The Complete Reference, Mc Graw Hill, 4th Edition 7. Byron Gottfried, Schaum's Outline of Programming with C, McGraw-Hill

4. GeeksforGeeks C Programming Language: GeeksforGeeks is an excellent online resource with tutorials, examples, and practice problems.

5. Codecademy Learn C: Codecademy offers an interactive learning platform with a C programming course.

6. Stack Overflow C Tag: Stack Overflow is a valuable resource for finding solutions to specific programming questions. The C tag is particularly useful for C-related queries.

Conclusion

- The stock market simulator presented in the provided code offers a functional and user-friendly interface for simulating a stock.
- It employs a modular structure, intuitive displays, and basic transaction handling.
- However, while the system achieves its primary objectives, there are areas for potential improvement and expansion.
- To further enhance the system, considerations for scalability, additional user features, and optimization can be addressed.
- Dynamically managing resource limits, incorporating localization for diverse language support, and implementing security measures could contribute to a more robust application.
- Furthermore, optimizing the code and introducing unit testing would ensure its reliability and maintainability over time. .